Phase 1 静态 FJSP 数学建模

FJSP-AGV-RL Research Platform

2026 年 5 月 31 日

1 范围说明

本文档描述当前仓库在 Phase 1 中实现的静态柔性作业车间调度（Flexible Job Shop Scheduling Problem, FJSP）数学模型。当前阶段只关注静态 FJSP，不包含 AGV 运输、路径规划、动态事件、强化学习或图神经网络。

2 集合与下标

符号	含义
J	工件集合，下标为 j
O_j	工件 j 的有序工序集合，下标为 o
M	机器集合，下标为 m
A_{jo}	工序 (j,o) 的可选机器集合

对于每个工件 j，工序必须满足工艺路线顺序：(j,o) 必须先于 $(j,o+1)$ 加工。

3 参数

符号	含义
p_{jom}	工序 (j,o) 在机器 m 上的加工时间
H	保守的时间上界

当前 CP-SAT baseline 使用如下保守 horizon：

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

这个上界不一定紧，但对当前小规模 baseline 是安全的。

符号	含义
$x_{jom} \in \{0,1\}$	若工序 (j,o) 分配到机器 m，则取 1
$S_{jo} \geq 0$	工序 (j,o) 的开始时间
$C_{jo} \geq 0$	工序 (j,o) 的完成时间
$C_{\max} \geq 0$	最大完工时间，也就是 makespan
$y_{abm} \in \{0,1\}$	同一机器上两个工序 a,b 的先后顺序变量

4 决策变量

这里 a,b 表示类似 (j,o) 的工序编号。经典 MIP 写法可以显式使用 y 变量；当前 OR-Tools 实现则使用 optional interval 和 `NoOverlap` 表达机器容量约束。

5 目标函数

目标是最小化 makespan：

$$\min C_{\max}.$$

6 约束

6.1 机器选择约束

每道工序必须且只能选择一台可行机器：

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

不在 A_{jo} 中的机器不是合法选择，代码中不会为这些机器生成候选分配。

6.2 加工时间连接约束

如果工序 (j,o) 选择机器 m，则完成时间必须等于开始时间加加工时间：

$$C_{jo} \geq S_{jo} + p_{jom} - H(1 - x_{jom}), \qquad \forall j,o,\ m \in A_{jo}.$$

6.3 工序顺序约束

同一工件内部必须满足工艺路线顺序：

$$C_{jo} \leq S_{j,o+1}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

6.4 机器容量约束

每台机器同一时刻最多只能加工一道工序。对任意两个不同工序 $a = (j,o)$ 和 $b = (k,q)$，若二者都可以选择机器 m，则需要满足：

$$S_a + p_{am} \leq S_b + H(1 - y_{abm}) + H(2 - x_{am} - x_{bm}),$$

$$S_b + p_{bm} \le S_a + H y_{abm} + H(2 - x_{am} - x_{bm}).$$

当两个工序都分配到机器 m 时，上述约束会强制二者不重叠。当前 CP-SAT 代码中，这部分由每台机器上的 `NoOverlap` 约束完成。

6.5 Makespan 定义

makespan 必须不小于每个工件最后一道工序的完成时间：

$$C_{\max} \ge C_{j,last(j)}, \qquad \forall j \in J.$$

7 Feasibility Checker 对应关系

代码中的 feasibility checker 会独立验证：

- 每道工序恰好出现一次；
- 工件编号和工序编号不越界；
- 每道工序选择的机器属于可选机器集合；
- 调度记录中的加工时间与实例数据一致；
- 同一工件内部满足工序顺序；
- 同一机器上的工序时间区间不重叠；
- 记录的 makespan 等于所有工序完成时间的最大值。

8 Dispatching Rule

严格 dispatching rule 共享候选元组：

$$(j, o, m, r_{jo}, s_{jom}, f_{jom}, p_{jom}),$$

其中 r_{jo} 是工件 ready time，s_{jom} 是在机器 m 上的最早可行开始时间，$f_{jom} = s_{jom} + p_{jom}$ 是候选完成时间。

规则	工序选择	机器选择
FIFO	最小 (r_{jo}, j, o)	最早完成机器
SPT	最小 p_{jom} 的 ready pair	与 pair 同时确定
EFT	最小 f_{jom} 的 ready pair	与 pair 同时确定
MWKR	剩余路线工作量最大	最早完成机器
MOPNR	剩余工序数最大	最早完成机器
Random	随机 ready pair	与 pair 同时确定

MWKR 的剩余路线工作量定义为：

$$W_{jo} = \sum_{q=o}^{last(j)} \min_{m \in A_{jq}} p_{jqm}.$$

MOPNR 使用剩余工序数作为 ready job 的优先级：

$$N_{jo} = last(j) - o + 1.$$

9 实验结果记录格式

Phase 1 实验脚本会在 `experiments/results/` 下输出标准 CSV。每一行表示一个算法在一个实例上的一次运行：

字段	含义
`experiment_id`	实验编号
`instance_name`	实例名称
`algorithm`	算法或求解器名称
`makespan`	目标值；若未返回调度则为空
`feasible`	feasibility checker 的结果，PASS 或 FAIL
`runtime_s`	运行时间，单位为秒
`note`	求解器状态、错误信息或验证说明
`objective_bound`	CP-SAT 给出的目标下界，若可用
`gap`	相对 gap，定义见下节，若可用
`solver_wall_time`	CP-SAT 内部 wall time，若可用

CSV 记录层只负责保存实验事实，不改变算法行为，也不改变可行性定义。Phase 1 实验脚本使用共用 runner helper，由 helper 统一负责计时、调用求解器、运行独立 feasibility checker、收集失败信息，并将可选的 solver metadata 写入 CSV 记录。

9.1 Benchmark 批量实验入口

批量实验入口会按照稳定路径顺序递归发现 `*.fjs` 文件。对于每个成功加载的静态 FJSP 实例，入口会运行所选的 Phase 1 启发式算法，并默认运行 CP-SAT 参考求解器。CP-SAT 的单实例 time limit 可以配置，从而限制较大 benchmark 集合上的计算时间。若文件格式错误，入口会保留一条 `LOAD` 算法、`FAIL` 状态的记录，避免损坏文件从实验报告中静默消失。

9.2 已跟踪的 FJSPLib 子集

仓库会跟踪一个较小的真实数据子集。文件来自 ScheduleOpt 官方 FJSPLib 仓库，并按照其 CC-BY-SA-4.0 许可证保留来源说明：`https://scheduleopt.github.io/benchmarks/fjsplib`。该子集覆盖三个来源系列和逐步增大的实例规模。

上表中的本地 CP-SAT 结果均通过独立 feasibility checker，并由求解器报告为 `OPTIMAL`。FJSPLib BKS 列来自上游参考数据，不是本仓库自行推断的值。

实例	工件数	机器数	FJSPLib BKS	本地 CP-SAT
`fattahi1`	2	2	66	66
Hurink edata `ft06`	6	6	55	55
Brandimarte `mk01`	10	6	40	40

10 CP-SAT 诊断信息

对于 CP-SAT 结果，代码会在通用的 `ScheduleResult` 对象中保存三个可选诊断字段：

$$LB = \text{BestObjectiveBound}, \qquad gap = \frac{C_{\max} - LB}{\max(1, C_{\max})}.$$

此外还会记录求解器内部的 wall time。对于 toy instance 上已证明最优的解，下界等于 makespan，因此 gap 为 0。启发式算法没有这些求解器诊断信息，对应 CSV 字段保持为空。